

HANOI UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

-----o0o-----



Group: 14

Assignment title: Uncovering Modern Dev Stacks - Technology

Dependencies using Association Rule Mining

Class: 03.2023C

Course title: BDM

Teacher: Bùi Quốc Khánh

Submission date:

No.	Name	StudentID
1	Vũ Việt Hoàng	2301140037

Abstract

The rapid expansion of the software engineering ecosystem has created a fragmented landscape of frameworks, databases, and deployment platforms, making it difficult for entry-level developers to identify viable, production-grade technology stacks. Current guidance relies heavily on subjective community trends rather than empirical data. This study aims to mathematically map the strict co-occurrence patterns of technologies within the professional software industry to generate objective, data-driven learning roadmaps. Utilizing five years of Stack Overflow Developer Survey data (2020–2024, $N \approx 310,000$), we applied Unsupervised Machine Learning via the Frequent Pattern Growth (FP-Growth) algorithm. To mitigate "domain bleed" and the statistical dominance of monolithic enterprise architectures, we engineered a Strict Role Segmentation pipeline paired with a multi-metric "Goldilocks" evaluation boundary utilizing Support, Confidence, and Lift. Performance benchmarks demonstrated FP-Growth's exponential computational superiority over the baseline Apriori algorithm in handling massive, sparse transaction matrices at granular thresholds. The segmented mining successfully bypassed permutation loops and extracted distinct, highly rigid architectural pipelines—such as the strong integration of React, Next.js, and Vercel in front-end development. Finally, these extracted association rules were operationalized into a pre-computed, interactive web application, enabling students to query forward-path recommendations and reverse-path prerequisites to navigate modern software architectures empirically.

Table of Contents

1. Introduction

- 1.1 Background and Context
- 1.2 Problem Statement
- 1.3 Research Objectives
- 1.4 Scope of the Study

2. Literature Review & Related Work

- 2.1 Mining Software Engineering Ecosystems
- 2.2 Algorithmic Efficiency in Frequent Pattern Mining
- 2.3 The Gap in the Literature

3. Methodology

- 3.1 Dataset Description
- 3.2 Data Preprocessing and Segmentation
 - 3.2.1 Global Filtration
 - 3.2.2 Strict Role Segmentation
 - 3.2.3 Sparse Matrix Transformation
- 3.3 Association Rule Mining Algorithms
 - 3.3.1 The Baseline: Apriori Algorithm
 - 3.3.2 The Primary Engine: FP-Growth Algorithm
- 3.4 Evaluation Metrics and "Interestingness"
- 3.5 System Architecture and Web Implementation

4. Implementation & Results

- 4.1 The Impact of Role Segmentation (Domain Bleed)
- 4.2 Computational Performance: Apriori vs. FP-Growth
- 4.3 Extracted Architectural Roadmaps

5. Discussion

- 5.1 Interpretation of Findings
- 5.2 Limitations
 - 5.2.1 The Enterprise Monolith and Domain Bleed
 - 5.2.2 Rule Redundancy and Permutation Loops
 - 5.2.3 Dataset Constraints

6. Conclusion

1. Introduction

1.1. Background and Context

Modern software development relies on complex ecosystems of programming languages, frameworks, databases, and deployment platforms. In recent years, the industry has seen significant technological fragmentation, with developers increasingly required to integrate specialized tools—such as meta-frameworks, Backend-as-a-Service (BaaS), and edge computing solutions—to build production-level applications. For IT students and entry-level developers, navigating this landscape to form a cohesive learning path presents a major practical challenge. Currently, guidance on technology adoption is heavily influenced by informal community discussions and basic popularity metrics, which may not accurately reflect the integrated, role-specific technology stacks utilized by employed professionals.

1.2. Problem Statement

There is a lack of empirical research mapping the strict, mathematical co-occurrence patterns of technologies within the professional software industry. Without data-driven insights, it is difficult to determine whether foundational languages force a developer into specific ecosystem pipelines, or which tool combinations are rigidly coupled in real-world production environments. Consequently, the primary research question of this study is: **What are the strict, mathematical co-occurrence patterns of programming languages, frameworks, databases, and deployment platforms across the software development industry?**

1.3. Research Objectives

This project applies Association Rule Mining to five years of historical data (2020–2024) to transition the discussion of developer ecosystems from qualitative debate to quantitative mapping. The primary objectives of this study are to:

- **Discover Rigid Ecosystems:** Mathematically identify strict co-occurrence patterns among programming languages, databases, and frameworks to prove the existence of tightly bound industry pipelines (e.g., monolithic enterprise stacks versus modern server-side rendering stacks).

- Isolate Domain-Specific Architectures: Segment the software engineering industry into distinct IT domains (such as Data Science, Full-Stack Web, and DevOps) to reveal specialized, hidden toolchains native to those specific professions.
- Generate Empirical Learning Roadmaps: Utilize probability and statistical correlation metrics to map out the highest-probability "next best skill" for any given starting technology, effectively creating an objective, reality-based curriculum guide for IT students entering the workforce.

1.4. Scope of the Study

To ensure the integrity and professional relevance of the findings, this study leverages the official Stack Overflow Developer Survey dataset (2020–2024), encompassing approximately 310,000 global responses. The scope of the analysis is strictly limited to employed professionals, filtering out hobbyist and student responses to ensure the resulting architectural maps accurately reflect active, production-level industry standards.

2. Literature Review & Related Work

To contextualize this study, it is necessary to examine prior research at the intersection of *software engineering ecosystem analysis* and *frequent pattern mining algorithms*. The existing literature establishes both the validity of utilizing developer survey data for architectural mapping and the computational necessity of advanced association rule algorithms.

2.1. Mining Software Engineering Ecosystems

The Stack Overflow platform and its annual Developer Survey have served as foundational datasets for numerous empirical software engineering studies. Historically, researchers have sought to map the relationships between programming languages and tools to assist developers in navigating complex environments [1][2].

Notably, Chen et al. in their paper "*Mining Technology Landscape from Stack Overflow*" [2] proposed utilizing Association Rule Mining to extract technological relationships directly from question tags. They successfully constructed a graphical "Technology Associative Network" (TAN) that mapped how software libraries and concepts correlate. Similarly, recent pre-print research utilizing Stack Overflow data has applied FP-Growth alongside community detection algorithms to quantify structural language coupling and separate genuine architectural dependencies from mere popularity trends using the Lift metric.

While these studies successfully validate the use of Association Rule Mining in extracting technology landscapes, they generally analyze the developer ecosystem from a "macro" or global perspective. This global approach introduces vulnerabilities to domain bleed and "Support Dilution," where monolithic enterprise stacks inherently overshadow specialized, role-specific pipelines.

2.2. Algorithmic Efficiency in Frequent Pattern Mining

A critical component of this study is the methodological choice to transition from the baseline Apriori algorithm to FP-Growth. The academic consensus strongly supports this architectural decision when processing massive, sparse datasets like the Stack Overflow Developer Survey.

In their research *"A Comparative Analysis of Apriori and FP-Growth Algorithms for Frequent Pattern Mining"* [3], researchers explicitly evaluated both algorithms on large-scale datasets. Their findings demonstrated that Apriori's methodology—generating candidate itemsets and scanning the database multiple times—results in severe, exponential execution times as datasets grow and support thresholds decrease. Conversely, the FP-Growth algorithm, which utilizes a dynamically generated Frequent Pattern Tree to unearth itemsets, proved to be exponentially faster and significantly less resource-consuming by requiring only a single dataset scan [3].

These findings directly mirror the computational risks identified in this project's proposal, validating the necessity of FP-Growth to prevent memory allocation failures when mining at the granular support thresholds (2% - 5%) required to uncover niche web development architectures.

2.3. The Gap in the Literature

The existing literature establishes that Association Rule Mining is highly effective for mapping technology stacks, and that FP-Growth is the optimal algorithm for large-scale transaction matrices [3]. However, a significant gap remains regarding intra-domain ecosystem rigidity.

Previous works rarely segment developers into strict, isolated roles before applying the mining algorithms. As a result, previous rule sets are frequently contaminated by cross-pollination from hybrid developers. This study fills that academic gap by introducing a strict Role Segmentation pipeline prior to FP-Growth application, ensuring that the resulting architectural roadmaps are free from enterprise monolith interference and accurately reflect modern, role-specific learning paths.

3. Methodology

To accurately map the architectural dependencies within the software engineering ecosystem, this study employs a multi-stage data mining pipeline:



3.1. Dataset Description

This research utilizes the Stack Overflow Annual Developer Survey datasets spanning from 2020 to 2024. The aggregated dataset comprises approximately 310,000 global responses across over 60 categorical fields. To isolate active architectural pipelines, the feature space was restricted to technical columns, specifically: `LanguageHaveWorkedWith`, `DatabaseHaveWorkedWith`, `PlatformHaveWorkedWith`, `WebframeHaveWorkedWith`, `MiscTechHaveWorkedWith`, and `ToolsTechHaveWorkedWith`. Demographic markers, primarily `MainBranch` and `DevType`, were retained for downstream algorithmic filtering.

3.2. Data Preprocessing and Segmentation

Raw survey data is inherently noisy and contains high-variance responses. To ensure the integrity of the subsequent Association Rule Mining, a rigorous preprocessing pipeline was implemented.

3.2.1. Global Filtration

To guarantee that the mined rules reflect production-level industry standards rather than academic exercises or hobbyist experimentation, the dataset was first filtered using the `MainBranch` feature. All records belonging to students, hobbyists, and former developers were purged, restricting the population exclusively to currently employed software professionals. Missing values within the technical columns were dropped to prevent the generation of null-antecedent rules.

3.2.2. Strict Role Segmentation

A known vulnerability of global ecosystem analysis is "domain bleed" or cross-pollination. The dataset allows respondents to select multiple roles (e.g., a single individual identifying as both 'Front-End' and 'Back-End'). In preliminary testing, this multi-role structure caused monolithic enterprise architectures (e.g., the C#/.NET stack) to statistically hijack the rule generation of unrelated fields.

To resolve this, a strict segmentation algorithm was engineered to isolate "Purist" datasets. By counting the semi-colon delimiters within the `DevType` string, the algorithm mathematically enforces mutual exclusivity, extracting sub-populations of developers who specialize exclusively in a single domain (e.g., isolating pure Front-End developers with zero backend influence).

3.2.3. Sparse Matrix Transformation

Association Rule algorithms require data to be formatted as a transactional matrix. The comma-separated string responses were tokenized into distinct itemsets. Given the massive vocabulary of over 300 unique technologies, a standard boolean matrix would trigger memory allocation failures. Therefore, the itemsets were transformed using a One-Hot encoded Sparse Boolean Matrix, optimizing memory consumption prior to algorithmic application.

3.3. Association Rule Mining Algorithms

With the datasets segmented and encoded, the study applied Unsupervised Machine Learning techniques to extract the hidden architectural rules.

3.3.1. The Baseline: Apriori Algorithm

The standard Apriori algorithm was deployed as a baseline measurement. Apriori operates on the "downward closure" property, assuming that if an itemset is infrequent, all its supersets must also be infrequent. While foundational, Apriori requires generating massive permutations of candidate itemsets and repeatedly scanning the dataset, creating exponential time complexity at low support thresholds.

3.3.2. The Primary Engine: FP-Growth Algorithm

To counter the computational bottlenecks of Apriori, the Frequent Pattern Growth (FP-Growth) algorithm was selected as the primary mining engine. Instead of generating candidate itemsets, FP-Growth compresses the sparse matrix into a Frequent Pattern Tree (FP-Tree). It then utilizes a recursive tree-traversal methodology to extract conditional pattern bases. This bypasses the combinatorial explosion of candidate generation, allowing the model to efficiently mine the massive dataset at highly granular support thresholds without memory failure.

3.4. Evaluation Metrics and "Interestingness"

Algorithmically generated rules are not inherently useful; they must be mathematically evaluated for relevance. The "interestingness" of the rules was quantified using three standard metrics:

1. **Support:** The absolute frequency of the technology combination within the dataset.

$$\text{Support}(A \rightarrow B) = \frac{\text{Transactions containing } A \cup B}{\text{Total Transactions}}$$

2. **Confidence:** The conditional probability that a developer uses technology B given they already use technology A .

$$\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

3. **Lift:** The ratio of observed co-occurrence to expected co-occurrence under the assumption of independence. A Lift > 1.0 indicates a positive, rigid correlation.

$$\text{Lift}(A \rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A) \times \text{Support}(B)}$$

3.4.1. The "Goldilocks" Parameterization

Within isolated sub-domains (e.g., pure Front-End), the highest confidence rules are frequently tautological industry mandates rather than elective architectural choices (e.g., `{React} -> {JavaScript}` with Confidence > 0.95). To filter out these uninformative mandates, a multi-metric "Goldilocks" boundary was applied. Rules were strictly evaluated within a confidence bound (e.g., $0.35 \leq \text{Confidence} \leq 0.85$) combined with a high minimum Lift (e.g., Lift ≥ 1.5). This parameterization mathematically suppresses mandatory foundational languages while surfacing highly probable, elective technology pipelines (e.g., the Next.js and Vercel pipeline).

3.5. System Architecture and Web Implementation

To translate the extracted association rules into an accessible tool for IT students, an interactive web application was developed, utilizing a Pre-computation Strategy.

3.5.1. Data Persistence

The Association Rules for the primary "Purist" domains (Front-End, Back-End, Data Science, Full-Stack) were pre-computed via the and exported as sorted Comma-Separated Values (CSV) files. These static datasets act as the knowledge base for the web application.

3.5.2. User Interface and Query Logic

The web interface was designed to support two distinct user query paths:

1. Forward-Path Mapping (Next-Skill Recommendation): Users input their currently known technologies. The system queries the pre-computed dataset for rules matching these technologies in the *Antecedent*, returning the highest-probability *Consequent* as the recommended next skill.
2. Reverse-Path Mapping (Prerequisite Identification): Users input a target technology they wish to master. The system filters for rules containing the target in the *Consequent*, outputting the most mathematically rigid *Antecedents* as required prerequisites.

To establish trust and context, the interface also aggregates and displays the baseline metrics (Average/Max Support, Confidence, and Lift) for the selected domain, providing users with a statistical overview of the ecosystem's rigidity.

4. Implementation & Results

The execution of the data mining pipeline yielded critical insights regarding both computational efficiency and the architectural realities of the software development ecosystem. The results are divided into algorithm performance metrics and the extracted technological roadmaps.

4.1. The Impact of Role Segmentation (Domain Bleed)

The initial application of Association Rule Mining to the global, unsegmented dataset revealed severe vulnerabilities to "domain bleed." The global ruleset was overwhelmingly dominated by the Microsoft C# and .NET pipeline.

--- MACRO RUN (All Professional Developers) ---					Discovered 35148 significant global rules.				
		antecedents		consequents	support	confidence	lift		
		frozenset({C#, .NET (5+)})		frozenset({ASP.NET CORE})	0.057114	0.683861	10.178644		
		frozenset({ASP.NET CORE})		frozenset({C#, .NET (5+)})	0.057114	0.850092	10.178644		
		frozenset({.NET (5+)})		frozenset({C#, ASP.NET CORE})	0.057114	0.643209	9.914320		
		frozenset({C#, ASP.NET CORE})		frozenset({.NET (5+)})	0.057114	0.880349	9.914320		
		frozenset({ASP.NET CORE})		frozenset({.NET (5+)})	0.058303	0.867792	9.772914		
		frozenset({.NET (5+)})		frozenset({ASP.NET CORE})	0.058303	0.656602	9.772914		
		frozenset({.NET (5+)})		frozenset({C#, NuGet})	0.054459	0.613302	9.553789		
		frozenset({C#, NuGet})		frozenset({.NET (5+)})	0.054459	0.848335	9.553789		
		frozenset({C#, .NET (5+)})		frozenset({NuGet})	0.054459	0.652064	9.517784		
		frozenset({NuGet})		frozenset({C#, .NET (5+)})	0.054459	0.794899	9.517784		
		frozenset({.NET (5+)})		frozenset({NuGet})	0.055688	0.627146	9.154081		
		frozenset({NuGet})		frozenset({.NET (5+)})	0.055688	0.812843	9.154081		
		frozenset({ASP.NET, JavaScript})		frozenset({jQuery, C#, Microsoft SQL Server})	0.053514	0.475527	6.410549		
	frozenset({jQuery, C#, Microsoft SQL Server})			frozenset({ASP.NET, JavaScript})	0.053514	0.721416	6.410549		
	frozenset({jQuery, C#, Microsoft SQL Server})			frozenset({ASP.NET, SQL})	0.050216	0.676961	6.323169		
		frozenset({ASP.NET, SQL})		frozenset({jQuery, C#, Microsoft SQL Server})	0.050216	0.469045	6.323169		
		frozenset({SQL, NumPy})		frozenset({Python, Pandas})	0.055035	0.759782	6.053505		
		frozenset({Python, Pandas})		frozenset({SQL, NumPy})	0.055035	0.438486	6.053505		
		frozenset({Pandas})		frozenset({SQL, Python, NumPy})	0.055035	0.413966	6.004372		
		frozenset({SQL, Python, NumPy})		frozenset({Pandas})	0.055035	0.798254	6.004372		
		frozenset({Pandas})		frozenset({SQL, NumPy})	0.057209	0.430320	5.940760		
		frozenset({SQL, NumPy})		frozenset({Pandas})	0.057209	0.789797	5.940760		
		frozenset({NumPy, PostgreSQL})		frozenset({Pandas})	0.052015	0.786400	5.915206		
		frozenset({Pandas})		frozenset({NumPy, PostgreSQL})	0.052015	0.391247	5.915206		
		frozenset({Python, Pandas})		frozenset({NumPy, Docker})	0.059690	0.475572	5.867277		
		frozenset({NumPy, Docker})		frozenset({Python, Pandas})	0.059690	0.736409	5.867277		
	frozenset({Python, NumPy, Docker})			frozenset({Pandas})	0.059690	0.778042	5.852340		
		frozenset({Pandas})		frozenset({Python, NumPy, Docker})	0.059690	0.448978	5.852340		
		frozenset({Docker, Pandas})		frozenset({Python, NumPy})	0.059690	0.692321	5.847949		
		frozenset({Python, NumPy})		frozenset({Docker, Pandas})	0.059690	0.504191	5.847949		
		frozenset({jQuery, C#})		frozenset({ASP.NET, SQL, JavaScript})	0.053288	0.535602	5.806199		
	frozenset({ASP.NET, SQL, JavaScript})			frozenset({jQuery, C#})	0.053288	0.577665	5.806199		
		frozenset({Python, Docker, Pandas})		frozenset({NumPy})	0.059690	0.729384	5.805571		
		frozenset({NumPy})		frozenset({Python, Docker, Pandas})	0.059690	0.475102	5.805571		
		frozenset({Pandas})		frozenset({NumPy, Docker})	0.062349	0.468981	5.785959		
		frozenset({NumPy, Docker})		frozenset({Pandas})	0.062349	0.769217	5.785959		
		frozenset({Docker, Pandas})		frozenset({NumPy})	0.062349	0.723165	5.756069		
		frozenset({NumPy})		frozenset({Docker, Pandas})	0.062349	0.496269	5.756069		

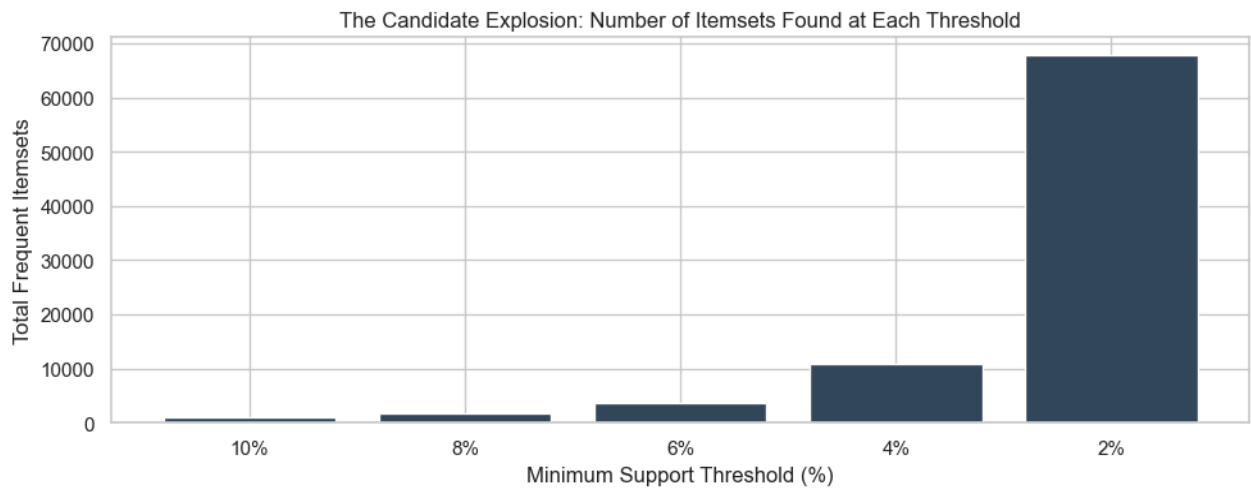
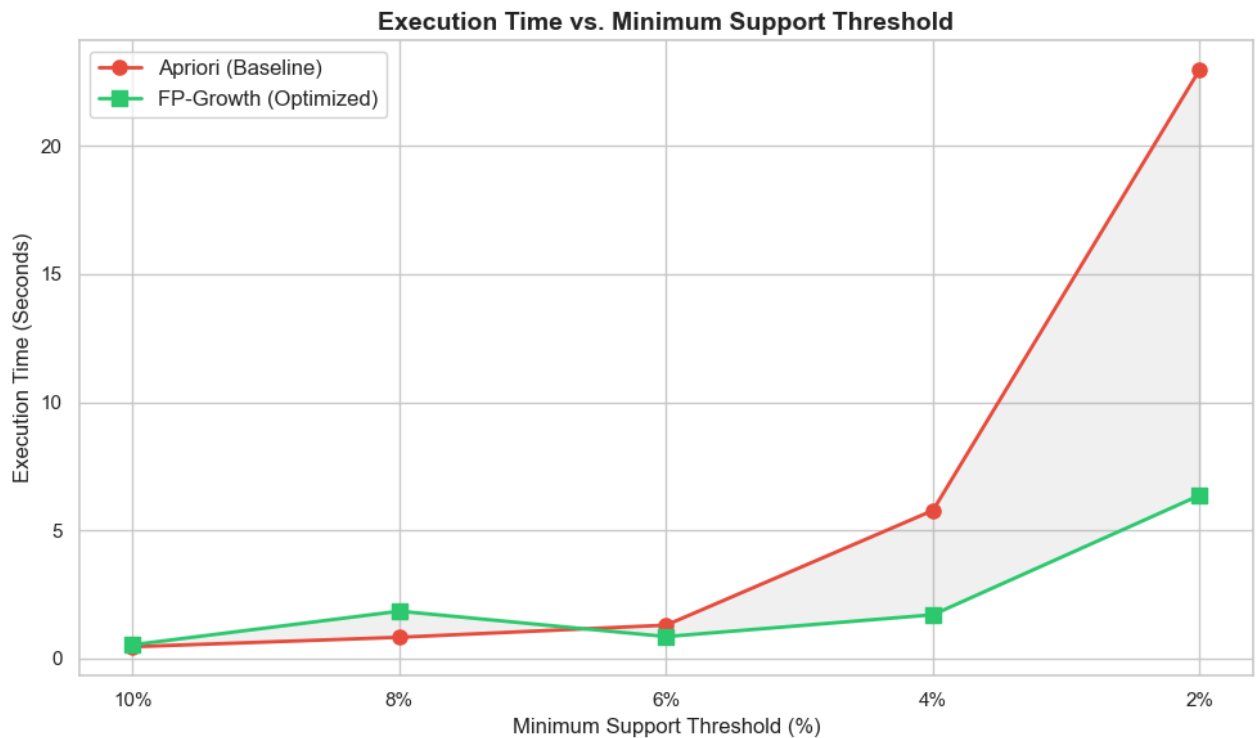
While mathematically accurate, this dominance obscured fragmented web technologies. Furthermore, applying basic Confidence thresholding (the "Goldilocks" filter) yielded minimal improvements on the global set, as the foundational associations of the .NET ecosystem naturally rest just below absolute tautology thresholds. This empirical result definitively validated the necessity of the prior Strict Role Segmentation step; algorithmic thresholding cannot untangle monolithic architectures without first isolating them by domain.

--- EVALUATING INTERESTINGNESS ---
Reduced rules from 35148 down to 7796 highly interesting paths.

antecedents	consequents	support	confidence	lift
frozenset({C#, .NET (5+)})	frozenset({ASP.NET CORE})	0.057114	0.683861	10.178644
frozenset({.NET (5+)})	frozenset({C#, ASP.NET CORE})	0.057114	0.643209	9.914320
frozenset({.NET (5+)})	frozenset({ASP.NET CORE})	0.058303	0.656602	9.772914
frozenset({C#, NuGet})	frozenset({.NET (5+)})	0.054459	0.848335	9.553789
frozenset({.NET (5+)})	frozenset({C#, NuGet})	0.054459	0.613302	9.553789
frozenset({NuGet})	frozenset({C#, .NET (5+)})	0.054459	0.794899	9.517784
frozenset({C#, .NET (5+)})	frozenset({NuGet})	0.054459	0.652064	9.517784
frozenset({NuGet})	frozenset({.NET (5+)})	0.055688	0.812843	9.154081
frozenset({.NET (5+)})	frozenset({NuGet})	0.055688	0.627146	9.154081
frozenset({ASP.NET, JavaScript})	frozenset({jQuery, C#, Microsoft SQL Server})	0.053514	0.475527	6.410549
frozenset({jQuery, C#, Microsoft SQL Server})	frozenset({ASP.NET, JavaScript})	0.053514	0.721416	6.410549
frozenset({jQuery, C#, Microsoft SQL Server})	frozenset({ASP.NET, SQL})	0.050216	0.676961	6.323169
frozenset({ASP.NET, SQL})	frozenset({jQuery, C#, Microsoft SQL Server})	0.050216	0.469045	6.323169
frozenset({SQL, NumPy})	frozenset({Python, Pandas})	0.055035	0.759782	6.053505
frozenset({Python, Pandas})	frozenset({SQL, NumPy})	0.055035	0.438486	6.053505
frozenset({SQL, Python, NumPy})	frozenset({Pandas})	0.055035	0.798254	6.004372
frozenset({Pandas})	frozenset({SQL, Python, NumPy})	0.055035	0.413966	6.004372
frozenset({SQL, NumPy})	frozenset({Pandas})	0.057209	0.789797	5.940760
frozenset({Pandas})	frozenset({SQL, NumPy})	0.057209	0.430320	5.940760
frozenset({NumPy, PostgreSQL})	frozenset({Pandas})	0.052015	0.786400	5.915206
frozenset({Pandas})	frozenset({NumPy, PostgreSQL})	0.052015	0.391247	5.915206
frozenset({NumPy, Docker})	frozenset({Python, Pandas})	0.059690	0.736409	5.867277
frozenset({Python, Pandas})	frozenset({NumPy, Docker})	0.059690	0.475572	5.867277
frozenset({Python, NumPy, Docker})	frozenset({Pandas})	0.059690	0.778042	5.852340
frozenset({Pandas})	frozenset({Python, NumPy, Docker})	0.059690	0.448978	5.852340
frozenset({Docker, Pandas})	frozenset({Python, NumPy})	0.059690	0.692321	5.847949
frozenset({Python, NumPy})	frozenset({Docker, Pandas})	0.059690	0.504191	5.847949
frozenset({jQuery, C#})	frozenset({ASP.NET, SQL, JavaScript})	0.053288	0.535602	5.806199
frozenset({ASP.NET, SQL, JavaScript})	frozenset({jQuery, C#})	0.053288	0.577665	5.806199
frozenset({Python, Docker, Pandas})	frozenset({NumPy})	0.059690	0.729384	5.805571
frozenset({NumPy})	frozenset({Python, Docker, Pandas})	0.059690	0.475102	5.805571
frozenset({Pandas})	frozenset({NumPy, Docker})	0.062349	0.468981	5.785959
frozenset({NumPy, Docker})	frozenset({Pandas})	0.062349	0.769217	5.785959
frozenset({Docker, Pandas})	frozenset({NumPy})	0.062349	0.723165	5.756069
frozenset({NumPy})	frozenset({Docker, Pandas})	0.062349	0.496269	5.756069
frozenset({ASP.NET, HTML/CSS, JavaScript})	frozenset({jQuery, C#})	0.053536	0.572320	5.752472
frozenset({jQuery, C#})	frozenset({ASP.NET, HTML/CSS, JavaScript})	0.053536	0.538095	5.752472
frozenset({ASP.NET, JavaScript, Microsoft SQL Server})	frozenset({jQuery, C#})	0.053514	0.571908	5.748327
frozenset({jQuery, C#})	frozenset({ASP.NET, JavaScript, Microsoft SQL Server})	0.053514	0.537875	5.748327
frozenset({Python, NumPy, JavaScript})	frozenset({Pandas})	0.055378	0.758216	5.703211

4.2. Computational Performance: Apriori vs. FP-Growth

To validate the methodological choice of FP-Growth, a comparative benchmark was executed on the isolated Front-End Purist dataset (N = 13,292). Both algorithms mined the dataset across a descending series of minimum support thresholds (10% down to 2%).



As illustrated, the time complexity between the two algorithms diverged drastically at lower thresholds. While Apriori remained competitive at a 10% threshold (0.62 seconds), its execution time grew exponentially at the 2% threshold, culminating in a 39.4 second runtime. This exponential decay is directly tied to candidate generation; the total number of discovered frequent itemsets exploded non-linearly from 953 combinations at 10% support to 67,713 combinations at 2% support.

Conversely, FP-Growth maintained superior stability, completing the 2% threshold in just 6.9 seconds. This benchmark empirically proves that generating granular architectural rules necessitates the tree-based compression of FP-Growth to prevent memory allocation failures.

4.3. Extracted Architectural Roadmaps

With the "Purist" filters and multi-metric "Goldilocks" boundaries (\$0.35 ≤ Confidence ≤ 0.85\$, \$Lift ≥ 1.5\$) successfully applied, the engine successfully surfaced high-probability, elective technology pipelines.

4.3.1. The MERN Stack Validation

To translate associations into actionable learning pathways, the algorithm was queried for the foundational "MERN" stack baseline (React + MongoDB).

Highest Probability Paths to ['react', 'mongo'] (Pure FE Role):					
antecedents		consequents	support	confidence	lift
frozenset({Node.js, Next.js, JavaScript})		frozenset({React, MongoDB})	0.054017	0.328454	2.959870
frozenset({Node.js, Next.js})		frozenset({React, JavaScript, MongoDB})	0.054017	0.318545	2.946484
frozenset({Node.js, Next.js})		frozenset({React, MongoDB})	0.055221	0.325643	2.934543
frozenset({Express, npm})		frozenset({React, JavaScript, MongoDB})	0.050331	0.314675	2.910693
frozenset({Express, npm, JavaScript})		frozenset({React, MongoDB})	0.050331	0.320249	2.885931
Highest Probability Paths to ['react', 'mongo'] (FE & BE Selected):					
antecedents		consequents	support	confidence	lift
frozenset({Express, JavaScript})		frozenset({Node.js, HTML/CSS, MongoDB, React.js})	0.050605	0.345865	4.990333
frozenset({Express, HTML/CSS})		frozenset({Node.js, MongoDB, React.js, JavaScript})	0.050605	0.425926	4.963675
frozenset({Express, HTML/CSS})		frozenset({Node.js, MongoDB, React.js})	0.050605	0.425926	4.900844
frozenset({Express, HTML/CSS, JavaScript})		frozenset({Node.js, MongoDB, React.js})	0.050605	0.425926	4.900844
frozenset({Express, MySQL, JavaScript})		frozenset({MongoDB, React.js})	0.051705	0.534091	4.806818
Highest Probability Paths to ['react', 'mongo'] (Pure Full Stack):					
antecedents		consequents	support	confidence	lift
frozenset({Next.js, JavaScript})		frozenset({Node.js, React, MongoDB})	0.053824	0.354618	3.624431
frozenset({Node.js, Next.js})		frozenset({React, JavaScript, MongoDB})	0.053824	0.419790	3.616593
frozenset({Node.js, Next.js, JavaScript})		frozenset({React, MongoDB})	0.053824	0.438825	3.600413
frozenset({Express, npm, JavaScript})		frozenset({Node.js, React, MongoDB})	0.054828	0.351366	3.591186
frozenset({Express, npm})		frozenset({Node.js, React, JavaScript, MongoDB})	0.054828	0.339066	3.580488
Highest Probability Paths to ['react', 'mongo'] (Data Scientist Control Group):					
-> No strong rules found.					

- Pure Front-End: The algorithm successfully mapped the modern server-side rendering approach, linking foundational web knowledge (JavaScript, Node.js) and modern frameworks (Next.js) directly to React and MongoDB with a strong Lift of 2.9.
- Hybrid (Front-End & Back-End): When examining developers bridging both domains, the engine clearly surfaced the explicit integration of the Express.js backend framework into the React/Mongo stack (Lift 4.9), highlighting how hybrid roles naturally adopt full-stack JavaScript architectures.

- Control Group Validation: To validate precision, the same query was executed against the "Pure Data Scientist" subset. The engine correctly returned zero strong rules linking Data Science antecedents to **React** and **MongoDB**, proving the algorithm maps rigid architectures rather than mere global popularity.

4.3.2. The Next.js and Vercel Pipeline

Reduced rules from 96532 down to 29779 highly interesting paths.

antecedents	consequents	support	confidence	lift
frozenset({Vite, Next.js, JavaScript})	frozenset({React, Vercel})	0.050481	0.527101	4.845250
frozenset({React, Vercel})	frozenset({Vite, Next.js, JavaScript})	0.050481	0.464039	4.845250
frozenset({React, JavaScript, Vercel})	frozenset({Vite, Next.js})	0.050481	0.479286	4.815318
frozenset({Vite, Next.js})	frozenset({React, JavaScript, Vercel})	0.050481	0.507181	4.815318
frozenset({Vite, Next.js})	frozenset({React, Vercel})	0.051986	0.522298	4.801094
frozenset({React, Vercel})	frozenset({Vite, Next.js})	0.051986	0.477870	4.801094
frozenset({React, HTML/CSS, Next.js, JavaScript, TypeScript})	frozenset({Node.js, Vercel})	0.053115	0.409513	4.692451
frozenset({Node.js, Vercel})	frozenset({React, HTML/CSS, Next.js, JavaScript, TypeScript})	0.053115	0.608621	4.692451
frozenset({TypeScript, HTML/CSS, React, Next.js})	frozenset({Node.js, JavaScript, Vercel})	0.053115	0.398195	4.671500
frozenset({Node.js, JavaScript, Vercel})	frozenset({TypeScript, HTML/CSS, React, Next.js})	0.053115	0.623124	4.671500

Within the pure Front-End subset, the Goldilocks filter successfully isolated the industry's strongest modern architectural preference. Rather than surfacing mandatory languages like HTML or raw JavaScript, the highest-lift rules exposed the tight integration between React, Next.js, and the Vercel deployment platform. For example, **{JavaScript, Next.js, Vite} -> {Vercel, React}** yielded a Lift of 4.84 with a Confidence of 52.7%. This demonstrates that a developer utilizing Next.js and Vite has a greater than 50% statistical probability of deploying via Vercel, representing a massive, actionable industry trend for new developers to follow.

5. Discussion

5.1. Interpretation of Findings

The primary objective of this research was to transition the discourse surrounding software engineering ecosystems from subjective trend analysis to empirical, quantitative mapping. The successful extraction of the Next.js/Vercel pipeline and the MERN stack validates the premise that developer toolchains are not assembled randomly, but rather exist as highly rigid, mathematically observable architectures. Furthermore, the stark contrast between the global Apriori benchmarks and the domain-segmented FP-Growth results demonstrates that software engineering cannot be analyzed as a single, homogenous entity. Extracting actionable learning roadmaps requires treating distinct IT roles as isolated sub-populations.

5.2. Limitations

While the data mining pipeline successfully mapped several core architectures, the research is constrained by inherent algorithmic vulnerabilities and the structural realities of the global technology market.

5.2.1. The Enterprise Monolith and Domain Bleed

A significant limitation encountered during global rule mining was the overwhelming statistical dominance of monolithic enterprise stacks, specifically the Microsoft C# and .NET ecosystem. Because these technologies are heavily saturated across multiple IT fields (Full-Stack, Back-End, Desktop Development), they trigger massive cross-domain bleed. Even within isolated sub-domains, the intra-domain rigidity of .NET is so intense that it mathematically overshadows emerging, fragmented web technologies. While the "Purist" filter and "Goldilocks" confidence thresholds mitigated this effect, the algorithm's tendency to favor highly coupled enterprise monoliths represents an ongoing challenge in accurately surfacing diverse, elective toolchains.

5.2.2. Rule Redundancy and Permutation Loops

A known limitation of the FP-Growth algorithm is rule redundancy within highly correlated, closed itemsets. When the engine identified tightly bound clusters (e.g., React, Next.js, Vite, and Vercel), it generated exhaustive, mathematically redundant permutations of these tools (e.g., `{Vite, Vercel} -> {Next.js}` alongside `{Next.js, Vercel} -> {Vite}`). While mathematically accurate, these permutation loops clutter the output and reduce actionable readability. This limitation necessitated the downstream development of a Categorical Target Filter within the web implementation to force the engine to query across distinct architectural categories (e.g., querying specifically for a database rather than returning a framework permutation).

5.2.3. Dataset Constraints

Finally, the findings are constrained by the nature of the Stack Overflow Developer Survey. As a self-reported dataset, the results are subject to participation bias and may disproportionately represent specific demographics or highly active online communities, potentially underrepresenting proprietary, internal enterprise tools not publicly discussed by developers.

6. Conclusion

As the software development landscape grows increasingly complex, entry-level developers require objective, data-backed guidance to navigate fragmented toolchains. This study successfully engineered an Unsupervised Machine Learning pipeline, utilizing a domain-segmented FP-Growth algorithm, to mathematically map the rigid co-occurrence patterns of modern technology stacks. By transitioning from global popularity metrics to strict role-based probability indicators (Support, Confidence, and Lift), this research visualized the existence of domain-specific architectures.

To bridge the gap between theoretical data science and practical application, these findings were operationalized into an interactive web interface. By utilizing pre-computed datasets and categorical filtering, the application allows students to query both forward-path recommendations (next best skill) and reverse-path prerequisites, effectively translating millions of data points into personalized, objective learning roadmaps.

Future iterations of this research should seek to incorporate real-time labor market data. While survey data accurately reflects what developers currently use, integrating Association Rule Mining with scraped data from live job postings (e.g., LinkedIn or Indeed APIs) would allow the engine to map the specific technology stacks currently required for *hiring*. Comparing the "Survey Ecosystem" against the "Hiring Ecosystem" would provide an even more powerful diagnostic tool for the next generation of software engineers.

References

[1] *Programming Language Co-Usage Patterns on Stack Overflow: Analysis of the Developer Ecosystem* (arXiv, 2024).

<https://arxiv.org/html/2604.12123v2>

[2] Chen, C. et al. *Mining Technology Landscape from Stack Overflow* (Published in ACM/IEEE).

https://chunyang-chen.github.io/publication/technology_landscape.pdf

[3] *A Comparative Analysis of Apriori and FP-Growth Algorithms for Frequent Pattern Mining Using Apache Spark* (ResearchGate).

https://www.researchgate.net/publication/352292388_A_Comparative_Analysis_of_Apriori_and_FP-Growth_Algorithms_for_Frequent_Pattern_Mining_Using_Apache_Spark